# ML Data Projects -

# Project 1= Data Annotation Quality Audit Pipeline

## README

# Data Annotation Quality Audit Pipeline

A simulated end-to-end pipeline for the kind of work ML data teams do :

generating labeled data, comparing annotators, and auditing quality —

similar to Amazon's ML Data Associate annotation and QA workflow.

## What this project does

1. **Generates a raw dataset** (300 items with captions and categories) — `generate_sample_data.py`

2. **Simulates two independent human annotators** labeling the same data with realistic accuracy levels — `simulate_annotators.py`

3. **Runs a quality audit** comparing both annotators against each other and against ground truth — `quality_audit.py`

## Key concepts demonstrated

- **Inter-annotator agreement** using Cohen's Kappa (industry-standard metric, corrects for chance agreement)

- **Quality auditing**: automatically flagging every disagreement into a review queue (`flagged_for_review.csv`)

- **Accuracy tracking** per annotator, similar to daily quality/productivity tracking in real annotation teams

- **SOP-style reporting**: a clean summary report an auditor or team lead could act on

## How to run

```bash
pip install pandas scikit-learn
python3 generate_sample_data.py     # creates raw_dataset.csv
python3 simulate_annotators.py      # creates annotated_dataset.csv
python3 quality_audit.py            # prints report + creates flagged_for_review.csv
```

## Sample output

```
Total items reviewed:           300
Annotator A accuracy:           91.00%
Annotator B accuracy:           83.33%
```

Raw agreement rate (A vs B): 74.67%

Cohen's Kappa (A vs B): 0.695

Interpretation: Substantial agreement

Items flagged for manual review: 76

` ` `

## How to extend this (ideas)

- Swap in a real image dataset (e.g. from Kaggle) and use actual bounding-box annotations instead of category labels

- Add a third annotator and calculate Fleiss' Kappa (for 3+ raters)

- Build a small Streamlit dashboard to browse flagged items visually

## Resume bullet you can use

"Built a data annotation quality-audit pipeline in Python that simulated multi-annotator labeling, computed inter-annotator agreement (Cohen's Kappa), and automatically flagged disagreements for review — reducing manual audit time by identifying the exact 76/300 records needing re-check."

## Code: generate_sample_data.py

```
"""
generate_sample_data.py
------------------------
This script CREATES a small fake dataset to annotate, so the project
is self-contained and doesn't require you to download external images.

It builds 300 "items" (simulating images) with:
- an item_id
- a text caption (simulating what an annotator would label)
- a "true" category (ground truth we pretend a second annotator agreed on)

In a real job, these would be actual image/audio/video files.
Here we simulate the DATA STRUCTURE so the annotation + quality-audit
logic below is realistic and resume-worthy.
"""

import random
import csv

random.seed(42)  # keeps results reproducible every time you run it

# Categories an annotator might tag images with (like Amazon's ML data teams do)
CATEGORIES = ["person", "vehicle", "animal", "furniture", "food", "electronics"]

# Some sample caption templates per category (simulates raw unlabeled data)
TEMPLATES = {
```

```
    "person": ["a man walking on the street", "a woman holding a phone", "a child playing
outside"],
    "vehicle": ["a red car parked outside", "a blue truck on the highway", "a bicycle near a
park"],
    "animal": ["a dog running in a field", "a cat sitting on a chair", "a bird flying in the
sky"],
    "furniture": ["a wooden chair in a room", "a sofa near a window", "a table with a lamp"],
    "food": ["a plate of pasta on a table", "a cup of coffee on a desk", "a bowl of fruit"],
    "electronics": ["a laptop on a desk", "a smartphone charging", "a television on a wall"],
}

rows = []
for i in range(1, 301):
    true_category = random.choice(CATEGORIES)
    caption = random.choice(TEMPLATES[true_category])
    rows.append({
        "item_id": f"IMG_{i:04d}",
        "caption": caption,
        "ground_truth_category": true_category
    })

with open("raw_dataset.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["item_id", "caption", "ground_truth_category"])
    writer.writeheader()
    writer.writerows(rows)

print("Created raw_dataset.csv with 300 items ready for annotation.")
```

## Code: simulate_annotators.py

```
"""
simulate_annotators.py
------------------------
In a real annotation job, TWO people label the same data independently,
and you compare their answers to measure annotation quality
(this is called "inter-annotator agreement").

Since this is a solo resume project, this script SIMULATES two annotators:
- Annotator A gets it right ~92% of the time (mimics a careful human)
- Annotator B gets it right ~85% of the time (mimics a slightly less careful human)

This produces a realistic annotated dataset with natural disagreements,
so the quality-audit script has real problems to catch.
"""

import csv
import random

random.seed(7)

CATEGORIES = ["person", "vehicle", "animal", "furniture", "food", "electronics"]

def simulate_annotation(true_label, accuracy):
    """Return the true label most of the time, otherwise a random wrong label."""
    if random.random() < accuracy:
        return true_label
    else:
        wrong_options = [c for c in CATEGORIES if c != true_label]
        return random.choice(wrong_options)
```

```
rows = []
with open("raw_dataset.csv", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        true_label = row["ground_truth_category"]
        annotator_a_label = simulate_annotation(true_label, accuracy=0.92)
        annotator_b_label = simulate_annotation(true_label, accuracy=0.85)
        rows.append({
            "item_id": row["item_id"],
            "caption": row["caption"],
            "ground_truth_category": true_label,
            "annotator_a": annotator_a_label,
            "annotator_b": annotator_b_label,
        })

fieldnames = ["item_id", "caption", "ground_truth_category", "annotator_a", "annotator_b"]
with open("annotated_dataset.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rows)

print("Created annotated_dataset.csv with two simulated annotators' labels.")
```

## Code: quality_audit.py

```
"""
quality_audit.py
------------------------
THIS IS THE CORE OF THE PROJECT.

It reads the annotated dataset (where two annotators labeled the same
300 items) and performs the kind of QUALITY AUDIT that an ML Data
Associate at Amazon would actually do:

1. Compares Annotator A and Annotator B on every item.
2. Calculates Cohen's Kappa - the industry-standard statistic for
   measuring how much two annotators agree (correcting for chance
   agreement, not just raw % match).
3. Compares each annotator against the "ground truth" to get individual
   accuracy scores.
4. Flags every disagreement into a CSV so a human reviewer/auditor can
   go check those specific items (this mirrors "raise failures/doubts
   in the portal" from the job description).
5. Prints a clean summary report.

Run this AFTER simulate_annotators.py has created annotated_dataset.csv
"""

import csv
from sklearn.metrics import cohen_kappa_score, accuracy_score

# ---------- STEP 1: Load the annotated data ----------
item_ids = []
captions = []
ground_truth = []
annotator_a = []
annotator_b = []
```

```
with open("annotated_dataset.csv", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        item_ids.append(row["item_id"])
        captions.append(row["caption"])
        ground_truth.append(row["ground_truth_category"])
        annotator_a.append(row["annotator_a"])
        annotator_b.append(row["annotator_b"])

total_items = len(item_ids)

# ---------- STEP 2: Calculate agreement metrics ----------

# Cohen's Kappa between the two annotators (measures agreement quality,
# NOT just how many they got "right" - it accounts for chance agreement)
kappa_score = cohen_kappa_score(annotator_a, annotator_b)

# Raw accuracy of each annotator against ground truth
accuracy_a = accuracy_score(ground_truth, annotator_a)
accuracy_b = accuracy_score(ground_truth, annotator_b)

# Simple agreement rate: how often A and B picked the SAME label
agreements = sum(1 for a, b in zip(annotator_a, annotator_b) if a == b)
agreement_rate = agreements / total_items

# ---------- STEP 3: Flag every disagreement for manual review ----------
flagged_rows = []
for i in range(total_items):
    if annotator_a[i] != annotator_b[i]:
        # Figure out which annotator (if either) was actually correct
        if annotator_a[i] == ground_truth[i]:
            note = "Annotator A correct, Annotator B likely error"
        elif annotator_b[i] == ground_truth[i]:
            note = "Annotator B correct, Annotator A likely error"
        else:
            note = "BOTH annotators incorrect - needs re-review"

        flagged_rows.append({
            "item_id": item_ids[i],
            "caption": captions[i],
            "annotator_a": annotator_a[i],
            "annotator_b": annotator_b[i],
            "ground_truth": ground_truth[i],
            "audit_note": note,
        })

# Save flagged items to their own CSV, like a real QA queue
fieldnames = ["item_id", "caption", "annotator_a", "annotator_b", "ground_truth",
"audit_note"]
with open("flagged_for_review.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(flagged_rows)

# ---------- STEP 4: Print a clean summary report ----------
print("=" * 55)
print("ANNOTATION QUALITY AUDIT REPORT")
print("=" * 55)
print(f"Total items reviewed:            {total_items}")
print(f"Annotator A accuracy:            {accuracy_a:.2%}")
print(f"Annotator B accuracy:            {accuracy_b:.2%}")
```

```
print(f"Raw agreement rate (A vs B):     {agreement_rate:.2%}")
print(f"Cohen's Kappa (A vs B):          {kappa_score:.3f}")

# Human-readable interpretation of the Kappa score (real QA teams use
# this exact scale - Landis & Koch, 1977)
if kappa_score > 0.80:
    interpretation = "Almost perfect agreement"
elif kappa_score > 0.60:
    interpretation = "Substantial agreement"
elif kappa_score > 0.40:
    interpretation = "Moderate agreement"
else:
    interpretation = "Poor agreement - annotation guidelines may need clarification"

print(f"Interpretation:                  {interpretation}")
print(f"Items flagged for manual review: {len(flagged_rows)}")
print(f"--> See 'flagged_for_review.csv' for the full list.")
print("=" * 55)
```

# Project 2: Text Data Quality & Grammar Validation Tool

## README

# Text Data Quality & Grammar Validation Tool

A rule-based + NLP-assisted tool for validating text data quality — the

kind of check an ML Data Associate performs on generated or transcribed

text before it's used as training data.

## What this project does

1. **Generates sample text data** with intentionally planted issues (grammar errors, extra whitespace, duplicates, casing problems, empty entries) — `sample_text_data.py`

2. **Runs rule-based quality checks** across 5 categories and flags every issue found — `text_quality_checker.py`

3. **Adds NLP-based linguistic analysis** using spaCy (tokenization, sentence splitting, named entity recognition, part-of-speech counts) — `spacy_analysis.py`

## Checks performed

- Empty / too-short entries

- Extra or leading/trailing whitespace

- Casing issues (ALL CAPS, missing capitalization)

- Duplicate text detection

- Common grammar mistakes (subject-verb agreement, wrong verb forms, apostrophe errors, commonly confused words like their/there/they're)

## How to run

```bash
pip install spacy
python -m spacy download en_core_web_sm

python3 sample_text_data.py       # creates raw_text_data.csv
python3 text_quality_checker.py   # creates quality_report.csv + prints summary
python3 spacy_analysis.py         # creates spacy_analysis_report.csv (bonus NLP layer)
```

## Sample output

```
Total samples checked:    15
Clean samples:           3 (20.0%)
Flagged samples:         12 (80.0%)

Sample of flagged issues:
  [TXT_002] Sentence does not start with a capital letter; Missing apostrophe: 'dont' should be "don't"
  [TXT_007] Duplicate of TXT_001
  [TXT_012] Incorrect verb form: 'have went' should be 'have gone'
  [TXT_013] Subject-verb agreement: plural subject needs 'were', not 'was'
```

## How to extend this (ideas)

- Swap in `language_tool_python` for broader, more powerful grammar checking (needs internet/Java)
- Add a Streamlit UI so flagged text can be reviewed and corrected in a browser
- Connect it to a real dataset (e.g. product reviews, subtitle files, or chatbot transcripts)

## Resume bullet you can use

"Built a Python text-quality validation tool that automatically detects
grammar errors, formatting inconsistencies, and duplicate entries in
text datasets, flagging 80% of a sample batch with specific,
explainable issues — incorporating spaCy for supplementary linguistic
analysis (POS tagging, entity recognition)."

## Code: sample_text_data.py

```
"""
sample_text_data.py
------------------------
Creates a small CSV of text samples (like captions, reviews, or
transcriptions an ML Data Associate might have to verify).

On purpose, some entries have REAL problems:
- grammar errors
- inconsistent casing
- extra whitespace
- duplicate entries
- very short/empty entries

This gives the quality-checker script (text_quality_checker.py)
real issues to catch, instead of a "perfect" dataset with nothing to find.
```

```
"""

import csv

samples = [
    ("TXT_001", "The quick brown fox jumps over the lazy dog."),
    ("TXT_002", "she dont like going to the market on sundays"),          # grammar error, no
capital
    ("TXT_003", "This   product   has  extra   spaces   between words."), # extra whitespace
    ("TXT_004", "HE WENT TO THE STORE YESTERDAY."),                       # all caps (style
issue)
    ("TXT_005", "The weather is nice today, isn't it?"),
    ("TXT_006", "me and him went to the park"),                           # grammar error
    ("TXT_007", "The quick brown fox jumps over the lazy dog."),          # exact duplicate of
TXT_001
    ("TXT_008", "he don't know the answer to this question"),             # grammar error
    ("TXT_009", ""),                                                      # empty entry
    ("TXT_010", "ok"),                                                    # too short, low
info
    ("TXT_011", "Their going to there house over they're."),              # classic
their/there/they're error
    ("TXT_012", "I have went to the store already."),                     # grammar error
(have went)
    ("TXT_013", "The results was very good this quarter."),               # subject-verb
agreement error
    ("TXT_014", "Please review this document before   the meeting."),     # extra whitespace
    ("TXT_015", "This is a perfectly well-formed sentence."),
]

with open("raw_text_data.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["text_id", "text"])
    writer.writerows(samples)

print(f"Created raw_text_data.csv with {len(samples)} text samples (includes intentional
issues).")
```

## Code: text_quality_checker.py

```
"""
text_quality_checker.py
------------------------
THIS IS THE CORE OF THE PROJECT.

This tool checks a batch of text data for the kind of QUALITY ISSUES
an ML Data Associate is expected to catch when validating generated
or transcribed text (per the job description: "apply strong language
skills, grammar knowledge, and linguistic rules to ensure generated
text adheres to proper grammar, syntax, and appropriate language usage").

It runs several independent checks on every text sample:

1. EMPTY / TOO SHORT      -> flags blank or near-blank entries
2. EXTRA WHITESPACE       -> flags double spaces, leading/trailing spaces
3. CASING ISSUES          -> flags ALL CAPS or missing capitalization at start
4. DUPLICATE DETECTION    -> flags exact duplicate text entries
5. GRAMMAR ISSUES         -> uses a rule-based checklist of common English
                             grammar mistakes (subject-verb agreement,
                             common misused words, contractions, etc.)
```

```
                             This avoids needing an internet-connected
                             grammar API, so it works fully offline.

Each issue found gets logged with a reason, then everything is saved
to 'quality_report.csv' and a written summary is printed.
"""

import csv
import re

# ---------- Rule-based grammar checks ----------
# Each rule is a (regex pattern, human-readable explanation) pair.
# This mimics how real annotation SOPs define "known bad patterns" to check.
GRAMMAR_RULES = [
    (r"\bdont\b", "Missing apostrophe: 'dont' should be \"don't\""),
    (r"\bdon't\b.*\bhe\b|\bhe\b.*\bdon't\b", "Subject-verb agreement: use 'doesn't' with
'he'"),
    (r"\bme and \w+ (went|did|saw|had|made)\b", "Pronoun order/case: consider '<Name> and I'
instead of 'me and <name>'"),
    (r"\bhave went\b", "Incorrect verb form: 'have went' should be 'have gone'"),
    (r"\bwas\b.*\b(results|items|things|people)\b|\b(results|items|things|people)\b.*\bwas\b",
"Subject-verb agreement: plural subject needs 'were', not 'was'"),
    (r"\btheir\b.*\bthere\b.*\bthey're\b|\bthere\b.*\btheir\b", "Possible their/there/they're
confusion - verify correct usage"),
]

def check_empty_or_short(text):
    if len(text.strip()) == 0:
        return "Empty text entry"
    if len(text.strip()) < 3:
        return "Text too short to be meaningful"
    return None

def check_whitespace(text):
    if "  " in text:  # two or more consecutive spaces
        return "Extra whitespace found between words"
    if text != text.strip():
        return "Leading or trailing whitespace found"
    return None

def check_casing(text):
    stripped = text.strip()
    if not stripped:
        return None
    letters_only = "".join(c for c in stripped if c.isalpha())
    if letters_only and letters_only.isupper() and len(letters_only) > 3:
        return "Entire text is in ALL CAPS - check if intentional"
    if stripped[0].islower():
        return "Sentence does not start with a capital letter"
    return None

def check_grammar_rules(text):
    issues = []
    lower_text = text.lower()
    for pattern, explanation in GRAMMAR_RULES:
        if re.search(pattern, lower_text):
            issues.append(explanation)
    return issues


# ---------- Main pipeline ----------
```

```python
def run_quality_check(input_file="raw_text_data.csv", output_file="quality_report.csv"):
    rows = []
    with open(input_file, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            rows.append(row)

    seen_texts = {}
    report_rows = []

    for row in rows:
        text_id = row["text_id"]
        text = row["text"]
        issues_found = []

        # Run each independent check
        empty_issue = check_empty_or_short(text)
        if empty_issue:
            issues_found.append(empty_issue)

        whitespace_issue = check_whitespace(text)
        if whitespace_issue:
            issues_found.append(whitespace_issue)

        casing_issue = check_casing(text)
        if casing_issue:
            issues_found.append(casing_issue)

        issues_found.extend(check_grammar_rules(text))

        # Duplicate check (compare against everything seen so far)
        normalized = text.strip().lower()
        if normalized and normalized in seen_texts:
            issues_found.append(f"Duplicate of {seen_texts[normalized]}")
        elif normalized:
            seen_texts[normalized] = text_id

        status = "FLAGGED" if issues_found else "CLEAN"
        report_rows.append({
            "text_id": text_id,
            "text": text,
            "status": status,
            "issues": "; ".join(issues_found) if issues_found else "",
        })

    # Save the full report
    with open(output_file, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["text_id", "text", "status", "issues"])
        writer.writeheader()
        writer.writerows(report_rows)

    return report_rows


if __name__ == "__main__":
    report = run_quality_check()

    total = len(report)
    flagged = sum(1 for r in report if r["status"] == "FLAGGED")
    clean = total - flagged
```

```
    print("=" * 55)
    print("TEXT DATA QUALITY REPORT")
    print("=" * 55)
    print(f"Total samples checked:    {total}")
    print(f"Clean samples:            {clean} ({clean/total:.1%})")
    print(f"Flagged samples:          {flagged} ({flagged/total:.1%})")
    print("-" * 55)
    print("Sample of flagged issues:")
    for r in report:
        if r["status"] == "FLAGGED":
            print(f"  [{r['text_id']}] {r['issues']}")
    print("-" * 55)
    print(f"Full report saved to: quality_report.csv")
    print("=" * 55)
```

## Code: spacy_analysis.py

```
"""
spacy_analysis.py
------------------------
BONUS SCRIPT - demonstrates familiarity with a real NLP library (spaCy),
which is commonly used in ML data/annotation pipelines for tasks like:
- Part-of-speech tagging (identifying nouns, verbs, adjectives)
- Detecting sentence structure
- Named entity recognition (people, places, organizations)

This adds an extra layer of automated linguistic analysis on top of the
rule-based checks in text_quality_checker.py.

Run this AFTER text_quality_checker.py has created quality_report.csv
"""

import csv
import spacy

# Load the small English model (downloaded via: python -m spacy download en_core_web_sm)
nlp = spacy.load("en_core_web_sm")

def analyze_text(text):
    """Run spaCy analysis and return useful linguistic info."""
    if not text.strip():
        return {"num_tokens": 0, "num_sentences": 0, "entities": "", "pos_summary": ""}

    doc = nlp(text)

    num_tokens = len(doc)
    num_sentences = len(list(doc.sents))

    # Named entities found (people, places, organizations, dates, etc.)
    entities = ", ".join(f"{ent.text}({ent.label_})" for ent in doc.ents)

    # Count how many nouns, verbs, adjectives appear (useful for judging
    # sentence richness/complexity - relevant for text quality review)
    pos_counts = {}
    for token in doc:
        if token.pos_ in ("NOUN", "VERB", "ADJ"):
            pos_counts[token.pos_] = pos_counts.get(token.pos_, 0) + 1
    pos_summary = ", ".join(f"{k}:{v}" for k, v in pos_counts.items())
```

```python
    return {
        "num_tokens": num_tokens,
        "num_sentences": num_sentences,
        "entities": entities,
        "pos_summary": pos_summary,
    }


if __name__ == "__main__":
    rows = []
    with open("raw_text_data.csv", newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            analysis = analyze_text(row["text"])
            rows.append({
                "text_id": row["text_id"],
                "text": row["text"],
                **analysis,
            })

    fieldnames = ["text_id", "text", "num_tokens", "num_sentences", "entities", "pos_summary"]
    with open("spacy_analysis_report.csv", "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)

    print("spaCy linguistic analysis complete.")
    print("Saved to: spacy_analysis_report.csv")
    print("\nSample output:")
    for row in rows[:5]:
        print(f"  [{row['text_id']}] tokens={row['num_tokens']}, "
              f"sentences={row['num_sentences']}, entities=[{row['entities']}]")
```